Odin OS: Cryptographically Authorized Memory Mutation in a RISC-V PMP-Enforced Agent Execution Environment

Nathan Walker
Ravenhelm Platform & Runestack Research
Email: nwalker85@gmail.com | ORCID: 0009-0005-0965-4424
Publication Candidate — Phase B.3 — November 3, 2026

Abstract

Current paradigms in autonomous artificial intelligence rely predominantly on probabilistic software guardrails—natural language prompts, output schemas, and user-space virtual sandboxes—to constrain agent behavior. These mechanisms are mathematically vulnerable to direct prompt injection, semantic jailbreaks, and memory-state corruption. In this paper, we present **Odin OS**, a bare-metal, zero-allocation (`#![no_std]`) 64-bit RISC-V microkernel that anchors autonomous agent trust boundaries directly in processor Physical Memory Protection (PMP) hardware comparators. Operating on the central axiom that **state which cannot prove its right to exist cannot be committed to memory or disk**, Odin OS enforces capability isolation via cache-line aligned (256-byte) binary tokens termed **TransitionDeeds**.

We present seven core contributions: (1) **Genuine Cryptographic Authorization**: Integration of RFC 8032 Ed25519 digital signature verification in pure `#![no_std]` without heap allocation (`no_alloc`), bound to a 32-byte pinned root public key; (2) **Complete Bi-Directional $W \oplus X$ Hardware Enforcement**: Strict domain partitioning guaranteeing user code is read-execute only (`R-X`, store attempts fault with `mcause = 7`), while user stack memory is read-write non-executable (`RW-`, instruction fetch attempts fault with `mcause = 1` and are redirected to caller return addresses); (3) **Dynamic Capability Revocation**: Privileged syscall interface (`SYS_REVOKE_LEASE`, `ecall 13`) that dynamically collapses PMP comparator windows to `OFF`; (4) **Deterministic 17-Case Adversarial Matrix**: Bare-metal QEMU verification demonstrating 100% hardware enforcement against fencepost violations, kernel overwrites, stack execution, and unprivileged CSR tampering; (5) **Machine-Checked Formal Verification**: Z3 SMT proofs (`QF_BV`) proving integer overflow immunity, memory disjointness, and anti-replay monotonicity; (6) **100% Mutation Testing Sensitivity**: Systematic injection of 15 architectural and cryptographic faults, with 15 out of 15 independently killed; and (7) **End-to-End LLM Bridge**: Live coupling to local `google/gemma-4-e4b` inference, demonstrating single-cycle authorized commits alongside instant hardware trapping of out-of-bounds prompt injections with zero bits committed to physical memory.

1 Introduction & Problem Statement

The rapid deployment of autonomous AI agents tasked with executing arbitrary code, modifying datastores, and manipulating abstract syntax trees (ASTs) has exposed an architectural vulnerability in modern computing. Contemporary agent runtimes treat safety as an inference-time alignment problem, attempting to prevent unauthorized state transitions through prompt engineering, output schemas, and container sandboxes.

This premise fails against adversarial attacks for three fundamental reasons:

1. **Prompt Invariance Failure**: No natural language instruction or heuristic system prompt is invariant against adversarial prompt injection. An adversarial token sequence can reliably cause a neural model to ignore its operational parameters.
2. **Semantic Escape**: In traditional operating systems, a compromised or hallucinating agent process running in user space possesses write access to its entire virtual address space. Once a store instruction is issued to the CPU, the underlying memory controller commits the mutated bytes regardless of agent intent.
3. **Post-Hoc Remediation**: Existing detection systems operate **after** memory has been mutated, relying on logging, rollbacks, or audit trails that incur latency and fail to prevent zero-day state corruption.

To resolve this vulnerability, we invert the trust model. Rather than attempting to guarantee that neural models will never produce malicious or errant store instructions, we construct a substrate where the physical CPU memory bus is physically incapable of asserting write-enable signals for unverified transactions.

2 Threat Model & Trust Assumptions

2.1 The Central Axiom & Security Invariant

Odin OS is constructed around a single deterministic axiom:

> ***"State that cannot prove its right to exist cannot be committed to memory or disk."***

This enforces the fundamental security invariant:

$$\text{Agent Request} \neq \text{Authority Grant}$$

2.2 Trust Boundary Classification

Components are strictly categorized across four trust boundaries:

- **CPU Hardware (PMP Comparators / CSRs)**: **Trusted**. Enforces address comparator matches, raises access faults, and enforces execution privilege modes.

- **M-Mode Microkernel (TCB)**: **Trusted**. Verifies Ed25519 signatures, programs PMP registers, and controls monotonic replay sequences.
- **Autonomous Agent (U-Mode Task)**: **Untrusted (Adversary)**. Executes untrusted model instructions and arbitrary arithmetic.
- **Host / Python Bridge**: **Untrusted Channel**. Transports model tokens and serializes deeds; assumed to be completely compromiseable.

3 TransitionDeed Protocol

State transitions in Odin OS are mediated through **TransitionDeeds**. A TransitionDeed is a standardized, 256-byte physical binary packet designed for cache-line alignment with zero heap allocations.

Offset	Field	Type	Description
0x00..0x02	magic	[u8; 2]	Magic identifier 0x59 0x47 (“YG”)
0x02..0x03	version	u8	Protocol specification version (0x02)
0x03..0x04	flags	u8	Capability flags (MUTATION, AST_STRICT)
0x04..0x08	sequence	u32	Monotonically increasing deed nonce
0x08..0x18	issuer_did	[u8; 16]	Cryptographic identity of the issuer
0x18..0x28	target_urn	[u8; 16]	Target entity uniform resource name
0x28..0x48	prev_deed_hash	[u8; 32]	Cryptographic hash of prior deed
0x48..0x68	base_digest	[u8; 32]	SHA-256 pre-mutation state digest
0x68..0x6C	lease_cycles	u32	Hardware execution cycle lifetime
0x6C..0x70	operation	u8 + u8 + u16	Opcode, escalation level, reserved
0x70..0x74	anchor_id	u32	Target AST Anchor identifier
0x74..0x78	target_offset	u32	Authorized buffer start offset
0x78..0x7C	target_length	u32	Authorized buffer span length
0x7C..0x9C	new_digest	[u8; 32]	Expected post-mutation state digest
0x9C..0xDC	signature	[u8; 64]	RFC 8032 Ed25519 digital signature
0xDC..0x100	padding	[u8; 36]	Zero padding to align to 256 bytes

Table 1: Physical layout of the 256-byte cache-line aligned TransitionDeed structure.

4 Kernel Architecture & PMP Enforcement

4.1 Privilege Domain Layout

Odin OS partitions physical memory into five strictly isolated privilege domains:

Domain	Physical Span	Mode	PMP Mode	Privilege & Architectural Purpose
Domain 0	0x10000000 .. 0x10001000	M-Mode	Default / ---	UART0 MMIO (Revoked from U-Mode)
Domain 1	0x80000000 .. 0x80010000	M-Mode	Default / ---	Privileged Kernel: Code, Stack, Keys, Traps
Domain 2	0x80010000 .. 0x80018000	U-Mode	TOR / R-X	Untrusted Agent Task Code ($W \oplus X$)
Domain 3	0x80018000 .. 0x80020000	U-Mode	TOR / RW-	Untrusted Agent Stack (Non-Executable)
Domain 4	0x80020000 .. lease_start	None	TOR / ---	Gated Pre-Lease AST Boundary
Domain 5	lease_start .. lease_end	U-Mode	TOR / RW-	Explicitly Authorized TransitionDeed Slice
Perimeter	All other physical addresses	None	OFF / ---	Hardware Default Deny (WE = 0)

Table 2: Hardened 7-Region PMP Layout and Privilege Partitioning.

4.2 Hardware W^X Stack Execution Prevention & Trap Handling

When user mode attempts to execute shellcode from the user stack (`0x80018100`), the CPU instruction fetch unit triggers an Instruction Access Fault (`mcause = 1`). The M-Mode trap handler intercepts the fault, records the invariant violation, and safely redirects `mepc` back to the return address `frame.ra` in `.user_text`:

```
if mcause == 1 && mepc >= USER_STACK_START as u64 && mepc < USER_STACK_END as u64 {
    unsafe { core::arch::asm!("csrw mepc, {0}", in(reg) frame.ra); }
    return;
}
```

This guarantees that stack execution attempts cannot escalate privileges or enter infinite fault loops.

4.3 Temporal Lease Revocation (`SYS_REVOKE_LEASE`)

To prevent temporal capability leakage, Odin OS introduces microkernel syscall 13 (`sys_revoke_lease`). When invoked, M-Mode clears active lease bounds and sets PMP Entry 4 to `OFF`. Subsequent store attempts to the previously valid lease slice immediately trigger a hardware Store Access Fault (`mcause = 7`).

5 Cryptographic Authorization

5.1 Canonical Domain Separation & Zero-Allocation Verifier

The signed message encompasses 172 contiguous bytes:

$$\text{Message} = \text{DOMAIN_SEPARATOR}_{16\text{B}} \,\|\, \text{Metadata}_{156\text{B}}$$

where `DOMAIN_SEPARATOR` is `"ODIN_DEED_V2\0\0\0\0"`. Signatures are verified against a 32-byte public key pinned in kernel read-only memory (`TRUSTED_ROOT_PUBLIC_KEY`). The verifier executes via `ed25519-compact` in pure `#![no_std]` with zero heap allocations, ensuring that signing private keys are never exposed to the agent environment.

6 Experimental Methodology

Our experimental evaluation executes on a 64-bit RISC-V platform (`riscv64gc-unknown-none-elf`) under QEMU 11.1.3 `virt`. Evidence is audited via a two-layer verification pipeline:

- **Layer A (In-Kernel)**: Direct M-Mode observation of trap causes, probed addresses, readback values, and memory preservation.
- **Layer B (External Host Verifier)**: An independent Python verifier (`tools/verify_evidence.py`) that strictly validates receipt schemas, enforces monotonic test sequences (1..17), and re-evaluates all invariant assertions independently of the kernel's claimed status.

7 Adversarial Results

#	Test Invariant Case	Target Addr	Width	Exp Fault	Obs Fault	mcause	Status
1	Exact Lower Bound	0x80020040	1B	No	No	0x00	**PASS**
2	Exact Upper Bound - 1	0x8002005F	1B	No	No	0x00	**PASS**
3	One Byte Below Lease	0x8002003F	1B	Fault	Fault	0x07	**PASS**
4	One Byte Above Lease	0x80020060	1B	Fault	Fault	0x07	**PASS**
5	Kernel Code Write	0x80000000	1B	Fault	Fault	0x07	**PASS**
6	Kernel Stack Write	0x8000FFF8	1B	Fault	Fault	0x07	**PASS**
7	User Code Write ($W \oplus X$)	0x80010000	1B	Fault	Fault	0x07	**PASS**
8	User Stack Execute ($W \oplus X$)	0x80018100	0B	Fault	Fault	0x01	**PASS**
9	Unmapped Perimeter Write	0x80030000	1B	Fault	Fault	0x07	**PASS**
10	Multi-byte Overlap Crossing	0x8002005E	4B	Fault	Fault	0x07	**PASS**
11	Forged Deed Issuer DID	0x00000000	0B	Reject	Reject	Syscall / 1	**PASS**
12	Replayed Deed Sequence	0x00000000	0B	Reject	Reject	Syscall / 2	**PASS**
13	Forged Deed Invalid Ed25519	0x00000000	0B	Reject	Reject	Syscall / 4	**PASS**
14	Deed Tampered Metadata Ed25519	0x00000000	0B	Reject	Reject	Syscall / 4	**PASS**
15	U-Mode CSR Access pmpcfg0	0x00000000	0B	Fault	Fault	0x02	**PASS**
16	Expired Lease Revocation	0x80020040	1B	Fault	Fault	0x07	**PASS**
17	U-Mode UART0 MMIO Write	0x10000000	1B	Fault	Fault	0x07	**PASS**

Table 3: Empirical receipts from the 17-case adversarial verification matrix (100% PASS).

7.1 Disambiguation of Multi-Byte Overlap Crossing (Test 10)

A 4-byte store at 0x8002005E overlaps the lease end (0x80020060). Because address 0x8002005E is aligned to 2 bytes but not 4 bytes ($0\text{x}5\text{E} \neq 0 \bmod 4$), we conducted experiments to isolate misaligned trap causes (mcause = 6) from PMP faults (mcause = 7). An unaligned 4-byte store entirely within the lease (0x80020052) succeeds without fault in QEMU virt, proving that the trap on 0x8002005E is strictly attributable to PMP comparator checking on the split transfer, aborting the transaction before commit.

8 Mutation Testing

To evaluate test suite sensitivity, we injected 15 deliberate architectural and cryptographic faults:

ID	Injected Architectural Mutation	Detecting Invariant Test	Result
M01	Bypass issuer DID check	Test 11 (FORGED_DEED_UNTRUSTED_ISSUER)	KILLED
M02	Disable Ed25519 signature verification	Tests 13 & 14 (FORGED_DEED_ED25519)	KILLED
M03	Permit sequence reuse (< instead of <=)	Test 12 (REPLAYED_DEED_SEQUENCE)	KILLED
M04	Wrapping addition on bounds (disable checked)	prop_deed_overflow_prevention	KILLED
M05	Expand PMP lease by 1 byte in setup_pmp	Test 4 (ONE_BYTE_ABOVE_HEAP)	KILLED
M06	Configure user code as RWX (0x0F)	Test 7 (USER_CODE_WRITE_DENIED)	KILLED
M07	Map UART MMIO in U-Mode	Test 17 (U_MODE_UART_MMIO_WRITE)	KILLED
M08	Corrupt register restore in trap_entry	Architectural Register Invariants	KILLED
M09	Skip PMP reprogramming on lease revocation	Test 16 (EXPIRED_LEASE_REVOCATION)	KILLED
M10	Tamper with receipt status emission	Dynamic Invariant Assertions	KILLED
M11	Mutate domain separator string	Host Test 8 (Cross-protocol)	KILLED
M12	Truncate canonical payload slice	Host Test 20 (Valid signature)	KILLED
M13	Corrupt pinned root public key	Matrix Test 1 & Host Test 20	KILLED
M14	Inject forged PASS into external Layer B trace	External Verifier (verify_evidence.py)	KILLED
M15	Omit stack execution trap redirection	Matrix Test 8 (Infinite loop)	KILLED

Table 4: Controlled mutation testing results demonstrating 100% detection sensitivity (15/15 killed).

9 Formal Assurance

Using the Z3 Theorem Prover version 5.3.1 (QF_BV), we machine-checked five formal SMT theorems:

1. **Integer Overflow Immunity**: Proved that checked addition in verify_authority mathematically guarantees $\text{end_offset} > \text{target_offset}$, preventing 32-bit wrap-around attacks (unsat).
2. **Privileged Memory Disjointness**: Proved that for all $2^{\{32\}} \times 2^{\{32\}}$ combinations of offset and length, an accepted capability lease is strictly disjoint from Kernel RAM, User Text, User Stack, and UART MMIO (unsat).
3. **Strict Buffer Containment**: Proved that all accepted leases remain strictly within AST_BUFFER_END (unsat).
4. **Identical Replay Impossibility**: Proved that resubmitting an already-consumed sequence number cannot be accepted by the microkernel (unsat).
5. **Stale Sequence Impossibility**: Proved that any sequence number $s \leq \text{last_sequence}$ cannot be accepted (unsat).

10 Limitations

1. **Hardware PMP Granularity**: In QEMU virt, PMP implements byte-exact Top-of-Range resolution ($G = 0$). Physical silicon implementations (e.g. StarFive JH7110) enforce non-zero grain sizes ($G \geq 4$), requiring 16-byte aligned capability allocations.
2. **Autonomous Timer Expiration**: Capability leases are revoked explicitly via microkernel syscalls (sys_revoke_lease); autonomous timer-interrupt cycle counting (mcycle) is designed for Phase C.
3. **Transactional State Atomicity**: PMP enforces spatial write boundaries; verifying that RAM contents bitwise match cryptographic digests (base_digest -> new_digest) requires shadow AST staging buffers.

11 Related Work

- **seL4 Microkernel**: Formal verification of capability-based microkernel isolation. Odin OS targets fine-grained tokenized AST leases for autonomous AI agents directly over bare-metal RISC-V PMP.
- **CHERI**: Fat-pointer capability hardware extensions. Odin OS enforces capabilities on standard, unmodified commodity RISC-V silicon without requiring custom ISA extensions.
- **Memory Protection Keys (Intel MPK)**: Hardware memory domain toggling. Odin OS isolates capability registers in privileged Machine Mode (M-Mode), preventing unprivileged User Mode tampering.

12 Future Hardware Directions

Phase C of the Odin OS research roadmap targets physical deployment on StarFive JH7110 RISC-V hardware, addressing store-buffer pipeline fence draining (`fence rw, rw`), non-volatile sequence persistence, and FPGA-based physical DRAM wordline gating controllers.

13 Conclusion

Odin OS establishes that autonomous AI agent security can be grounded in verifiable cryptographic authority and deterministic physical-layer enforcement. By implementing zero-allocation Ed25519 signature verification, enforcing strict bi-directional $W \oplus X$ protections, validating temporal capability revocation, and subjecting the microkernel to a 17-case adversarial matrix, 100% mutation testing, and machine-checked Z3 formal proofs, we achieve mathematical containment against prompt injection, model jailbreaks, and memory corruption.

14 Reproducibility & Artifact Availability

The complete Odin OS codebase, linker scripts, formal SMT specifications, and Layer B external verifiers are archived under permanent DOI on Zenodo.

- **Repository**: `https://github.com/nwalker85/odinos`
- **Target Triple**: `riscv64gc-unknown-none-elf`
- **External Verifier**: `python3 tools/verify_evidence.py`
- **Formal Proof Command**: `z3 formal/deed_bounds_verification.smt2 && z3 formal/sequence_anti_replay.smt2`